

Engineering review & product roadmap

A phased implementation guide for the development team
and a technical review packet for your professor.

38 FINDINGS

6 DELIVERY PHASES

EVIDENCE + GATES

Version 1.0 | 1 October 2026 | Repository snapshot 4ecc6a7

The engineering objective

Make every approval traceable to the user's actual requirement and relevant executed evidence.

The immediate priority

Close unsafe execution and false-approval mechanisms before increasing autonomy or expanding integrations.

The product milestone

A dependable, supervised product within a declared support scope, with independently demonstrated value.

01 / HOW TO USE THIS REVIEW

Read this before changing code

Decision today

Proceed with a supervised product-hardening effort. Do not treat the current PASS verdict or benchmark headline as a release gate. The five layers are a reasonable structure; the acceptance semantics and execution boundary need repair.

38 findings with code pointers, implementation steps, and completion tests. **7 critical-priority items** include one conditional on legacy-proxy deployment. **Six phases** progress from a regression baseline to a supervised beta.

Read / do first	Pages	Purpose
Evidence and current limitations	3-4	Understand what is established and why the old headline must be retired.
Product contract and architecture	5-7	Agree on initial scope, trust boundaries, and evidence schemas.
Issue register and detailed tickets	8-22	Turn each finding into a reviewed implementation task.
Phased delivery roadmap	23-27	Build in dependency order and meet the exit gate before advancing.
Tests and qualification metrics	28-29	Decide when the supported product is ready for a pilot.
Professor review and demonstrations	30-31	Prepare honest answers, controlled demos, and specific research questions.
Reproduction recipes and sources	32-33	Reproduce failures safely and close tickets with evidence.

First assignments for the junior developer

- Start QB-26 and QB-38: test harness, CI, and run/evidence schemas. Preserve the existing results; create a new branch for fixes.
- Reproduce and fix QB-01, QB-04, QB-07, and QB-08 with assertions. These are contained failures with large impact.
- Review the sandbox and protected-verifier design with a senior engineer before implementing the larger execution changes.

Do not start with a vector database, a larger default judge, more votes, or more agent integrations. Those choices do not resolve the known safety and false-approval mechanisms.

02 / REVIEW BASIS

Evidence, scope, and limitations

This report consolidates the earlier technical review and an expanded code inspection. The remote repository was checked again on **1 October 2026**; its HEAD remained **4ecc6a7**. Findings describe that snapshot, not unpushed local changes.

Evidence label	What it means
Reproduced	Observed in an isolated local fixture against the repository modules. Model, database, or HTTP responses were stubbed where necessary to isolate logic.
Saved artifact	Supported by committed benchmark JSON or logs. Generated Cue patches were not saved, so their real behavior was not independently rerun.
Code-confirmed	Directly supported by a specific implementation path. Production configuration or exposure was not assumed.
Design risk	A failure mode or missing guarantee inferred from the architecture. It requires calibration or a targeted experiment, not an invented incident.

Review performed

- Inspected L1-L5, root orchestration, benchmark execution/reporting, and website API code. Examined all **15** saved benchmark JSON files.
- Ran controlled probes for verdict aggregation, parsing, metadata overrides, change capture, memory, extraction, and API validation. Reproductions used temporary fixtures and mocked external services.
- Checked official Cursor, Devin, and SWE-agent documentation for the competitive discussion. Sources are linked in the final reference page.

Limits of this report

The six Claude/Ollama tasks were not rerun. The complete Cue checkout and generated patches were not inspected. No live website API penetration test, production secret/configuration audit, load test, or proof of exploitability on the deployed service was performed. No source fixes, email sends, or production changes were made.

Priority is tied to impact

Critical means unsafe execution or invalid acceptance can occur, or an upstream cost exposure exists if deployed. High blocks dependable operation or evidence. Medium limits supported scope, maintainability, or scaling. Severity is not a claim that exploitation occurred.

03 / THE MOST IMPORTANT EVIDENCE

The headline result is not defensible yet

The saved T-002 contract contains incorrect unit conversions. QB records three positive votes for each of these ACs, while the baseline is marked wrong. This is an error in the scoring target before sample size enters the discussion.

Saved expected result	Correct result	QB votes
30,000 ms becomes 5m	30s	true / true / true
120,000 ms becomes 20m	2m	true / true / true
25,000 ms becomes 4m 10s	25s	true / true / true

Evidence: [bench/results/T-002_1790654092867.json](#) : 34

The absent patch leaves two possibilities: the agent followed the wrong specification, or the judge approved a patch inconsistently with its criterion. Neither validates improved correctness. The report must not infer which happened.

Latest saved result for each task

Task	QB result	Attempts	Baseline
T-001	PASS	1	FAIL
T-002	PASS	1	FAIL
T-003	FAIL	3	FAIL
T-004	PASS	1	FAIL
T-005	PASS	2	FAIL
T-006	PASS	2	Not present

Recomputed: QB 5/6, baseline 0/5 available comparisons, 1.67 QB attempts on average, and about 441 seconds per task. These rows mix historical experimental conditions; they are a reconciliation, not a valid new benchmark.

About **67%** of the summed recorded QB runtime is L4. Even under ideal random-sampling assumptions, 5/6 has a roughly 44%-97% Wilson 95% interval. The actual task selection does not support that ideal assumption.

Keep existing results as development history. Do not advertise 83% versus 17%, 6/6, or +34pp as independently established product effectiveness. A newer unpublished run needs its own immutable evidence bundle.

04 / INITIAL PRODUCT CONTRACT

Define a product you can actually qualify

Recommended first release

A supervised CLI for bounded changes in supported JavaScript/TypeScript repositories, using one validated test runner and one coding-agent adapter. It produces an inspectable patch and an evidence report. A developer reviews the merge.

Dimension	Initial boundary
Agent	Claude Code only until other adapters pass the same contract tests. Keep model and CLI versions in every run manifest.
Runtime	One verified Linux sandbox execution profile. A macOS client may control it after install and cancellation smoke tests.
Checks	One reliable runner, such as node:test, plus explicit approved check commands/adapters. Expand the matrix after fixture coverage exists.
Task scope	Small, bounded code changes with testable behavior. Unsupported UI, database, infrastructure, or cross-service behavior is unresolved unless an appropriate check adapter is configured.
Output	Complete candidate changes, exact accepted requirements, executed evidence, unresolved items, time/cost data, and a reproducible decision.
Human role	Approve material assumptions and final merge. Escalations are measured; suppressing questions cannot compensate for silent requirement errors.

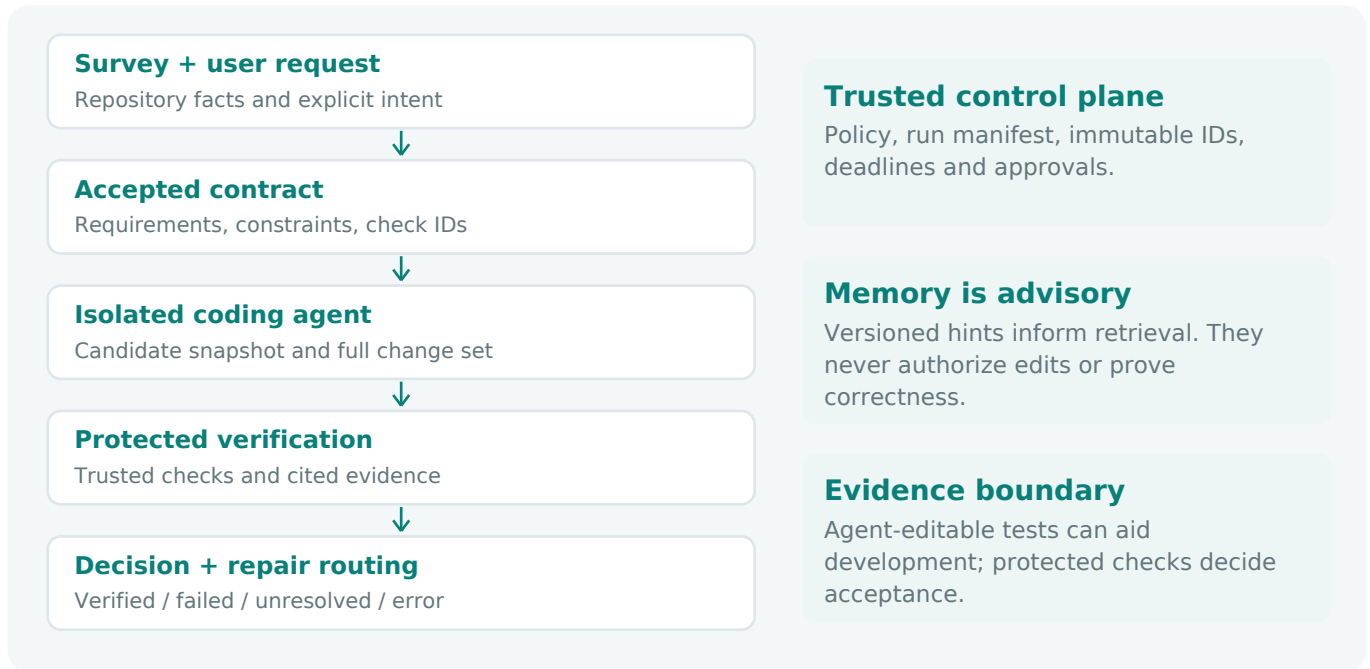
A working limited product has three properties

- **Dependable operation:** installation, execution, cancellation, failure recovery, artifact retrieval, and rollback all work within the declared support matrix.
- **Defensible decisions:** PASS is backed by relevant executed checks and declared policy. Missing evidence and unsupported behavior remain visible.
- **Measured user value:** on independently evaluated tasks, users spend less review/correction effort at an acceptable correctness rate and total cost.

These are proposed release criteria, not capabilities that the current repository already satisfies. General correctness, fully autonomous merging, broad language support, and a competitive moat require further evidence.

05 / TARGET ARCHITECTURE

Retain the layers; strengthen their contracts



Make one trusted run record span all five layers. L1 and L2 may iterate before finalizing the contract. The coding agent can propose changes and tests; the protected verifier owns acceptance. Memory supplies versioned hints and cannot elevate a judgment.

Outcome	Required meaning
VERIFIED	A valid, nonempty accepted contract exists; all mandatory checks and policies pass; no material evidence is missing.
FAILED	A required behavior or policy is contradicted by concrete evidence. Preserve other errors/unknowns in the report.
UNRESOLVED	No definitive approval is possible because required evidence is absent or insufficient. Route to evidence collection or a person.
ERROR	The agent, runner, model, or environment failed operationally. Do not mislabel it as an implementation verdict.
BLOCKED / CANCELLED / DRY_RUN	Separate lifecycle outcomes. None is a correctness claim. Track no_change separately from execution status.

Decision order: validate inputs; handle lifecycle state; evaluate definite policy/check failures; surface operational errors; surface missing evidence; approve only after all required checks pass. Store every component result so precedence never erases evidence.

06 / PROPOSED SCHEMAS AND INVARIANTS

Turn requirements into executable evidence

Introduce these interfaces before rewriting individual layers. Names below are design proposals. Keep application metadata trusted and immutable; allow model output only in explicitly semantic fields.

Record	Minimum contents
RunManifest	run_id, schema_version, QB revision, source repo identity/base SHA, agent/model versions, configuration hash, accepted contract hash, sandbox policy, timestamps, outcome.
Requirement	id, original source clause, origin (explicit / inferred / proposed), accepted meaning, risk, AC IDs, check IDs, unresolved decisions.
CheckSpec	id, requirement/AC IDs, approved adapter, typed parameters, fixture ID, expected outcome, timeout, required flag, trusted implementation hash.
CheckResult	check_id, status (pass / fail / error / not_run), exit code/signal, duration, evidence IDs, before/after results, runner version.
Attempt / Evidence	attempt number, candidate snapshot and patch hashes, execution status, commands/results, judge outputs, parent attempt, repair reason, immutable artifact references.

Example: cumulative VAD statistics

R-1: speechCount increases once per completed speech event. **R-2:** totalSpeechMs accumulates the durations. **R-3:** counters are local to each AdaptiveVAD instance. Map all three to controlled event-sequence checks.

Fixture	Expected observation
New instance; no events	{ speechCount: 0, totalSpeechMs: 0 }
Two completed events: 250 ms and 500 ms	{ speechCount: 2, totalSpeechMs: 750 }
Create a second instance	Its counters start at zero; the first instance retains its counters.

The exact event boundaries and duration semantics require confirmation against the actual VAD implementation. The example illustrates independent behavioral checks; it is not an invented specification to impose without review.

Invariant: every required source clause has an accepted disposition and evidence mapping. No empty contracts, duplicate IDs, unapproved semantic additions, arbitrary generated shell commands, or silent material truncation.

07 / 38 ACTIONABLE FINDINGS

Issue register at a glance

Click a finding to jump to its detailed ticket. Phase is the planned delivery stage; page is its implementation card. All findings are open at the reviewed snapshot. Critical* is conditional on the legacy proxy being deployed.

ID	Finding	Priority	Phase / page
QB-01	Repository filenames can execute shell commands	Critical	1 / 10
QB-02	Agent execution has no enforced containment	High	1 / 10
QB-03	Change capture omits parts of the actual result	High	1 / 10
QB-04	Model output can overwrite trusted metadata	High	1 / 11
QB-05	Benchmark reset is destructive and not reproducible	High	1 / 11
QB-06	Test-command failure can still produce PASS	Critical	2 / 11
QB-07	Loose response parsing reverses negative judgments	Critical	2 / 12
QB-08	An empty or clarification-only contract can pass	Critical	2 / 12
QB-09	Scope and constraints are not enforced as policy	High	2 / 12
QB-10	Failure routing loses test failures and repair actions	High	2 / 13
QB-11	Judge evidence is incomplete and lacks provenance	High	2 / 13
QB-12	Keyword signals claim facts that were not established	Medium	2 / 13
QB-13	L1 generates mathematically incorrect specifications	Critical	2 / 14
QB-14	The AC checklist omits the essential requested behavior	High	2 / 14
QB-15	Voting and preservation rules are not calibrated	High	2 / 14
QB-16	Verification plans are prose rather than executable checks	High	2 / 15
QB-17	Intent is finalized before repository grounding	High	2 / 15
QB-18	Context selection and graph depth do not match the claims	Medium	3 / 15
QB-19	Symbol extraction silently drops valid exports	Medium	3 / 16

07 / 38 ACTIONABLE FINDINGS

Issue register: runtime through delivery

Click a finding to jump to its detailed ticket. Phase is the planned delivery stage; page is its implementation card. All findings are open at the reviewed snapshot. Critical* is conditional on the legacy proxy being deployed.

ID	Finding	Priority	Phase / page
QB-20	Test discovery is mistaken for test coverage	Medium	3 / 16
QB-21	Deadlines, cancellation, and cost budgets are incomplete	High	3 / 16
QB-22	Execution errors are conflated with dry-run or no change	High	1 / 17
QB-23	Successful repairs never reach normal memory persistence	High	3 / 17
QB-24	Memory can mix repositories and opposite instructions	Medium	3 / 17
QB-25	Memory persistence lacks safe lifecycle and concurrency	Medium	3 / 18
QB-26	The project has no effective root regression gate	High	0 / 18
QB-27	The benchmark lets QB define its own scoring target	Critical	4 / 18
QB-28	Baseline resources and verification paths are unequal	High	4 / 19
QB-29	Latest-per-task aggregation mixes experiments	High	4 / 19
QB-30	Six tuned utility tasks do not establish effectiveness	High	4 / 19
QB-31	Supported adapters and onboarding are not productized	Medium	5 / 20
QB-32	Telemetry and privacy descriptions overstate anonymity	High	5 / 20
QB-33	Metrics can be forged with a known registration email	High	5 / 20
QB-34	Admin exports expose avoidable credential and CSV risks	High	5 / 21
QB-35	Signup input, delivery, and abuse handling are fragile	High	5 / 21
QB-36	Legacy proxy permits unauthenticated paid upstream calls	Critical*	1 / 21
QB-37	Deployment and public claims have drifted	High	5 / 22
QB-38	Runs lack a durable, versioned evidence record	High	0 / 22

08 / IMPLEMENTATION TICKETS

Findings 01-03

QB-01 Repository filenames can execute shell commands**Critical** | **Reproduced** | **Phase 1** | **Effort S**Source: [context/git.js](#) : 18

Problem. A repository filename is interpolated into an execSync shell string. Double quotes do not neutralize shell command substitution. A harmless filename probe created a marker during context collection, before any coding agent ran.

Implement. 1. Replace shell strings with execFile/spawn argument arrays and shell:false. 2. Preserve the -- path separator. 3. Audit all subprocess calls for interpolation and reject paths outside the selected repository.

Done when. A filename containing shell metacharacters is treated as a literal path; no marker is created. Spaces, quotes, newlines, and leading hyphens are handled without command execution.

QB-02 Agent execution has no enforced containment**High** | **Code / design** | **Phase 1** | **Effort L**Source: [agent/runner.js](#) : 54; [bench/baseline.js](#) : 17

Problem. Claude is started in the supplied repository with --dangerously-skip-permissions and inherited environment. Agent edits, repository scripts, and prompt injection can reach resources outside the intended task. Existing tests and package scripts are writable by the agent.

Implement. 1. Run in a disposable clone inside an OS/container sandbox. 2. Allow only required environment variables, credentials, mounts, and egress. 3. Keep policy, trusted tests, and grading outside the writable checkout; document the threat model.

Done when. A fixture attempting a host-file write, forbidden network request, and trusted-test modification is denied. The user checkout and its Git metadata remain unchanged. A senior engineer approves the boundary.

QB-03 Change capture omits parts of the actual result**High** | **Reproduced** | **Phase 1** | **Effort M**Source: [agent/runner.js](#) : 73; [bench/baseline.js](#) : 37

Problem. Bare git diff captures only unstaged tracked changes. New files, staged edits, and agent-created commits can be invisible. A fixture with a staged edit and a new file produced an empty captured diff.

Implement. 1. Pin a trusted base revision before execution. 2. Capture the complete final state in the isolated checkout, including untracked files and changes relative to that base. 3. Use NUL-safe path handling and record renames, deletions, binary changes, modes, and symlinks.

Done when. Fixtures covering staged, unstaged, new, committed, renamed, deleted, binary, and unusual-path changes are captured or explicitly rejected as unsupported. No partial capture can receive PASS.

08 / IMPLEMENTATION TICKETS

Findings 04-06

QB-04 Model output can overwrite trusted metadata

High | Reproduced | Phase 1 | Effort S

Source: [intent/compiler.js](#) : 108; [intent/schema.js](#) : 9; [qb.js](#) : 264

Problem. The compiler spreads model output after its own id, timestamp, and raw_request. A mocked response replaced the original request and set id to ../../model-chosen-location. The artifact path then resolved outside the intended contract directory.

Implement. 1. Parse only allowed semantic fields from the model. 2. Generate trusted metadata after validation and never accept it from model output. 3. Validate artifact identifiers and verify resolved output paths stay beneath the artifact root.

Done when. Injected id, repo_path, created_at, and raw_request values cannot override application metadata. Traversal identifiers cannot write outside the designated run directory.

QB-05 Benchmark reset is destructive and not reproducible

High | Code-confirmed | Phase 1 | Effort M

Source: [bench/run.js](#) : 218; [bench/tasks.json](#) : 2

Problem. The harness runs checkout and clean in a hard-coded working repository and swallows errors. It can discard a developer's changes, does not restore a pinned commit or staged state, and leaves ignored files and external state behind.

Implement. 1. Accept an explicit repository plus base revision. 2. Create a disposable checkout per arm and task. 3. Reset database/service/cache fixtures explicitly and fail on setup errors; never clean the supplied working tree.

Done when. Start with dirty tracked, staged, untracked, and ignored files in the source checkout. The harness preserves them all and produces identical initial snapshots for both arms, even after an agent commit.

QB-06 Test-command failure can still produce PASS

Critical | Reproduced | Phase 2 | Effort M

Source: [verify/checker.js](#) : 33; [verify/verifier.js](#) : 60

Problem. Nonzero exits and runner failures are converted into parsed output counts. A command exiting 2 yielded zero failed tests; affirmative judge responses then produced PASS. Missing runners, timeouts, and unrecognized formats are not distinct outcomes.

Implement. 1. Preserve exit code, signal, timeout, duration, and raw evidence. 2. Use a runner adapter with machine-readable results. 3. Treat runner error, missing required tests, and zero unexpected tests as verification errors that cannot approve a task.

Done when. Exit 2, timeout, missing executable, collection failure, malformed report, and zero-test fixtures never pass. A real failing test is distinguished from infrastructure failure.

08 / IMPLEMENTATION TICKETS

Findings 07-09

QB-07 Loose response parsing reverses negative judgments**Critical** | Reproduced | Phase 2 | Effort SSource: [verify/judge.js](#) : 194

Problem. The prose fallback searches for words such as implemented and true. The response "The criterion is not implemented." was parsed as true on all three controlled calls.

Implement. 1. Require a strict judgment schema with a boolean-or-null field and structured evidence references. 2. Delete keyword-to-verdict fallbacks. 3. Classify malformed output separately; allow at most a bounded format retry without turning it into evidence.

Done when. Negative prose, fenced garbage, arrays, null JSON, wrong types, and missing fields produce `invalid_judgment` or `unresolved`. None become a positive verdict through text guessing.

QB-08 An empty or clarification-only contract can pass**Critical** | Reproduced | Phase 2 | Effort SSource: [verify/verifier.js](#) : 30; [qb.js](#) : 75; [bench/run.js](#) : 115

Problem. `verify()` defaults missing criteria to an empty array. With a nonempty diff and no blocking tests, the empty array produces PASS. The benchmark does not handle a clarifying response; root handling also does not recheck a second clarification.

Implement. 1. Validate contract type and nonempty, unique requirements at every boundary. 2. Use an explicit `needs_clarification` state. 3. Prevent agent execution and verification until a finalized contract exists; expose that state cleanly in noninteractive mode.

Done when. Empty ACs, missing goal, duplicate IDs, a first clarification, and a second clarification never execute the agent or return PASS. Tests cover CLI and benchmark entry points.

QB-09 Scope and constraints are not enforced as policy**High** | Reproduced | Phase 2 | Effort MSource: [verify/checker.js](#) : 105; [verify/verifier.js](#) : 47; [agent/briefing.js](#) : 52

Problem. Scope violations are included in the report but do not block PASS. Constraints only appear in the briefing unless also represented as ACs. Relevant files are treated as an apparent allowlist even though retrieval confidence is not authorization.

Implement. 1. Separate `relevant_files` from `allowed_changes` and `protected_paths`. 2. Convert enforceable constraints into policy checks. 3. Block forbidden edits and require an explicit policy update for legitimate scope expansion.

Done when. A forbidden file change blocks PASS even when every judge vote is positive. A newly relevant file is not silently authorized, and each declared constraint maps to evidence or an explicit unresolved item.

08 / IMPLEMENTATION TICKETS

Findings 10-12

QB-10 Failure routing loses test failures and repair actions**High** | Reproduced | Phase 2 | Effort MSource: [verify/verifier.js : 54](#); [qb.js : 176](#); [bench/run.js : 168](#)

Problem. Failed tests only override a pass verdict. Failed tests plus unknown ACs produce partial. With all ACs true, failed tests produce fail but no AC failures or repair hints, so the loop stops. No baseline test run proves that a failure is pre-existing.

Implement. 1. Aggregate policy, execution, test, and semantic outcomes independently. 2. Create repair actions from concrete test failures. 3. Compare base and candidate test evidence and route infrastructure errors to environment recovery, not code repair.

Done when. A newly failing regression enters a bounded repair with its test evidence. A known base failure stays visible. Unknown ACs cannot hide a definite failure, and unchanged patches do not restart endless repairs.

QB-11 Judge evidence is incomplete and lacks provenance**High** | Code-confirmed | Phase 2 | Effort MSource: [verify/judge.js : 85](#); [verify/verifier.js : 44](#)

Problem. The judge sees one AC and the first 6,000 characters of diff, without the relevant unchanged implementation or runtime evidence. It can miss later integration files and cannot resolve vague ACs such as the correct string format.

Implement. 1. Retrieve criterion-specific source, callers, complete relevant hunks, and check results. 2. Assign immutable evidence IDs with file hashes and ranges. 3. Report truncation and missing evidence explicitly; never present partial input as complete.

Done when. A defect after character 6,000 and a defect inside an unchanged helper are visible through retrieval. Missing material evidence yields unresolved, with the missing artifact named.

QB-12 Keyword signals claim facts that were not established**Medium** | Reproduced | Phase 2 | Effort MSource: [verify/checker.js : 136](#)

Problem. A comment mentioning the target, a commented export, and an unrelated return statement set defined/exported/returns to true. Keyword scanning also examines deleted and context lines. These are text hints presented as high-confidence facts.

Implement. 1. Parse supported-language syntax and resolve the actual symbol and binding. 2. Keep heuristic matches labeled as search hints. 3. Do not use absence from added lines as evidence that an unchanged implementation is absent.

Done when. Comments, strings, deleted code, multiline exports, unrelated returns, and pre-existing functions receive correct classifications. Heuristic signals alone cannot satisfy behavioral ACs.

08 / IMPLEMENTATION TICKETS

Findings 13-15

QB-13 L1 generates mathematically incorrect specifications

Critical | Saved artifact | Phase 2 | Effort M

Source: [bench/results/T-002_1790654092867.json](#) : 34; [intent/compiler.js](#) : 90

Problem. The saved contract expects 30,000 ms -> 5m, 120,000 ms -> 20m, and 25,000 ms -> 4m 10s. QB records unanimous PASS votes on these ACs. The missing patch prevents determining whether the agent obeyed the bad spec or the judge misgraded it.

Implement. 1. Preserve original requirements and mark generated examples as proposals. 2. Validate computable examples independently. 3. Review unsupported semantic additions and freeze the accepted contract before execution; never alter it to fit a result.

Done when. The three arithmetic errors are rejected. A unit-conversion fixture is checked by trusted executable calculations. A human-approved test oracle is independent of the implementation-generating model.

QB-14 The AC checklist omits the essential requested behavior

High | Saved artifact | Phase 2 | Effort M

Source: [bench/results/T-005_1790656354435.json](#) : 19; [agent/briefing.js](#) : 38

Problem. T-005 asks for cumulative speech events and duration, but its ACs only require object shape, nonnegative integers, and exports. A constant-zero implementation satisfies those stated conditions while missing the requested behavior.

Implement. 1. Give each explicit request clause a stable requirement ID. 2. Map each requirement and constraint to one or more ACs and checks. 3. Flag uncovered requirements, duplicate ACs, and unsupported additions before the task is executable.

Done when. A constant-zero VAD fixture fails after simulated events. Duration accumulates over multiple events, resets per instance, and remains independent across instances. Every request clause has a traceable disposition.

QB-15 Voting and preservation rules are not calibrated

High | Design risk | Phase 2 | Effort M

Source: [verify/judge.js](#) : 18; [verify/judge.js](#) : 142

Problem. Identical low-temperature calls share blind spots and missing evidence. Repeated attempts can change the vote without changing code. Changing preservation ACs to null avoids a false repair but leaves partial, not verified success.

Implement. 1. Make preservation claims bounded to named tests/interfaces. 2. Calibrate verdicts against independently labeled patches and report abstention. 3. Cache evidence by patch hash and measure whether extra votes outperform one judge at equal cost.

Done when. An unchanged patch does not become approved by repeated resampling alone. False acceptance, false rejection, and abstention are measured separately. No rule treats null as proof of preservation.

08 / IMPLEMENTATION TICKETS

Findings 16-18

QB-16 Verification plans are prose rather than executable checks**High** | Code-confirmed | Phase 2 | Effort LSource: [agent/briefing.js : 148](#); [verify/checker.js : 23](#); [intent/schema.js : 18](#)

Problem. The plan is added to the coding briefing, but L4 runs a generic suite and model judgment. A plan item saying measure performance or test integration does not produce that evidence. Existing tests may not exercise the new behavior.

Implement. 1. Define a versioned check registry with approved adapters and typed parameters. 2. Map ACs to checks, expected results, and fixture IDs. 3. Execute checks in the protected verifier; do not execute arbitrary shell text emitted by L1.

Done when. Every required behavioral AC has an executed check or an explicit unresolved status. A plan item cannot be marked complete merely because it was printed or an agent claimed it ran.

QB-17 Intent is finalized before repository grounding**High** | Code-confirmed | Phase 2 | Effort MSource: [qb.js : 72](#); [intent/cli.js : 26](#); [intent/dsa.js : 9](#)

Problem. The root path compiles without repo context although the standalone intent CLI can supply it. Regex ambiguity rules can ask even when the request supplies the detail. Clarification is a single pass, and fallback fields can conceal incomplete output.

Implement. 1. Gather a lightweight repository survey before semantic compilation. 2. Distinguish explicit requirements, inferred constraints, and proposed defaults. 3. Support bounded clarification rounds and a machine-readable handoff state; reject incomplete final contracts.

Done when. Root and standalone compilation receive equivalent grounding. A fully specified cleaner request does not trigger a redundant question, and a still-ambiguous answer remains blocked with its unresolved choice.

QB-18 Context selection and graph depth do not match the claims**Medium** | Code-confirmed | Phase 3 | Effort MSource: [context/extractor.js : 168](#); [context/builder.js : 43](#)

Problem. Ranking primarily uses paths and filenames, then selects up to 25 files. Direct imports are recorded but the claimed two-level traversal is absent. Retries reuse the original context even after changed symbols or new files.

Implement. 1. Support one language family well and add content/symbol retrieval. 2. Implement bounded graph traversal with a visited set, explicit depth, and caller edges. 3. Refresh affected entries after each patch and expose omissions and retrieval budget.

Done when. A task whose relevant symbol lives in a generically named file is retrieved in a 200+ file fixture. A two-hop dependency and a new repair file are present; irrelevant files do not dominate the package.

08 / IMPLEMENTATION TICKETS

Findings 19-21

QB-19 Symbol extraction silently drops valid exports**Medium** | Reproduced | Phase 3 | Effort MSource: [context/extractor.js : 32](#); [context/extractor.js : 214](#); [context/builder.js : 71](#)

Problem. `module.exports = { alpha, beta }` returns only `alpha`. Symbol locations use first substring occurrence and names overwrite one another across files. Truncating reads at 8,000 characters can hide trailing definitions and exports.

Implement. 1. Use a syntax parser for the supported language. 2. Store qualified symbol IDs and actual source spans. 3. Index full supported files or record explicit limits; retrieve bounded snippets only when building prompts.

Done when. Single and trailing CJS exports, aliases, methods, duplicate names, comments, and late-file exports are indexed correctly. Each symbol resolves to the actual declaration or export, not a comment.

QB-20 Test discovery is mistaken for test coverage**Medium** | Reproduced | Phase 3 | Effort MSource: [context/extractor.js : 129](#); [context/extractor.js : 153](#); [context/detector.js : 61](#); [agent/briefing.js : 111](#)

Problem. Conventional `test_auth.py` and `auth_test.go` were not discovered. Matching filenames does not prove behavior is covered. Runner detection is heuristic, and some advertised languages/runners lack reliable execution and result parsing.

Implement. 1. Call the field `test_associations`, not `coverage`. 2. Allow explicit project check configuration and support only validated adapters. 3. Add standard naming conventions and consume actual coverage data only when available.

Done when. Supported-language fixtures discover the expected tests and run the configured command. Unsupported runners are clearly rejected. No UI or briefing claims code is covered merely because a test file exists.

QB-21 Deadlines, cancellation, and cost budgets are incomplete**High** | Code-confirmed | Phase 3 | Effort MSource: [verify/judge.js : 121](#); [context/builder.js : 188](#); [intent/compiler.js : 19](#); [qb.js : 291](#)

Problem. Ollama and telemetry fetches have no explicit application deadline. Sequential three-vote judging dominates saved runtime. Synchronous child processes block orchestration, and there is no overall token/cost/time budget or process-tree cancellation.

Implement. 1. Add per-call and total-run deadlines with cancellation propagation. 2. Use asynchronous execution with bounded local-model concurrency. 3. Record stage time, tokens when available, and agent usage; make telemetry bounded and nonblocking.

Done when. A hung mock model, stalled telemetry endpoint, and child process tree all terminate within configured limits. Artifacts survive cancellation, locks are released, and the final state identifies the interrupted stage.

08 / IMPLEMENTATION TICKETS

Findings 22-24

QB-22 Execution errors are conflated with dry-run or no change

High | Code-confirmed | Phase 1 | Effort M

Source: agent/runner.js : 65; verify/verifier.js : 33; qb.js : 176

Problem. Agent failure returns a null diff even if partial edits exist. The root verifier then reports no-diff and may say dry-run. A valid already-satisfied task is also indistinguishable from an agent that did nothing.

Implement. 1. Preserve execution status and capture partial changes even after failure. 2. Use separate dry_run, execution_error, cancelled, and no_change states. 3. Permit no-change success only when the original requirement is independently verified.

Done when. Timeout, authentication failure, nonzero exit, and a genuine dry-run are distinct. Partial edits remain inspectable. An already-satisfied fixture passes checks without fabricated changes.

QB-23 Successful repairs never reach normal memory persistence

High | Reproduced | Phase 3 | Effort M

Source: memory/index.js : 62; qb.js : 200; qb.js : 211

Problem. The root passes only the final report to memory. A successful report has no failed-AC repair hints, while remember() expects hints in a non-failing report. A controlled fail-then-pass sequence saved zero repairs.

Implement. 1. Keep an append-only attempt history with patch and evidence hashes. 2. Link each proposed repair to the subsequent patch and reevaluation. 3. Store observed resolution separately from a model suggestion and from independently confirmed success.

Done when. A failed AC repaired on attempt two creates one traceable resolved repair. An unchanged patch, abandoned task, or unrelated later success does not label the hint as proven.

QB-24 Memory can mix repositories and opposite instructions

Medium | Reproduced | Phase 3 | Effort M

Source: memory/store.js : 20; memory/scorer.js : 8; memory/index.js : 95

Problem. Paths /tmp/a/b and /tmp/a_b map to the same memory namespace. Removing not makes opposite deployment requests have similarity 1. Records lack revision compatibility checks, and file hints can come from failed runs.

Implement. 1. Use a collision-resistant repository identity plus schema version. 2. Preserve negation and task intent; treat lexical overlap as a hint only. 3. Validate referenced paths/revisions and rank trustworthy resolved outcomes separately from failures.

Done when. The two repositories never share records. Opposite-intent requests do not reuse actionable repairs automatically. Stale paths, traversal hints, incompatible revisions, and failed-run suggestions are clearly handled.

08 / IMPLEMENTATION TICKETS

Findings 25-27

QB-25 Memory persistence lacks safe lifecycle and concurrency**Medium** | Code-confirmed | Phase 3 | Effort MSource: [memory/store.js : 36](#); [memory/store.js : 75](#); [memory/sandbox/run.js : 14](#)

Problem. Each recall reads all records; stats use an unlocked read-modify-write. Corrupt lines are silently discarded. The sandbox sets QB_MEMORY_DIR after importing the module that captures it, so its intended temporary store may not be used.

Implement. 1. Inject the store path at construction and lock per repository. 2. Use atomic stats replacement and report corruption. 3. Add retention/compaction and bounded recall; retain JSONL until measured concurrency or query needs justify a database.

Done when. Concurrent fixture writes preserve counts, a truncated record is reported without losing valid data, and demo/tests write only beneath their test directory. Recall latency is measured at the supported history size.

QB-26 The project has no effective root regression gate**High** | Code-confirmed | Phase 0 | Effort MSource: [package.json : 10](#); [verify/sandbox/run.js : 1](#)

Problem. Root npm test prints usage and succeeds. Existing sandbox scripts largely demonstrate output, rather than asserting invariants. No checked-in CI workflow was found in the reviewed snapshot.

Implement. 1. Establish node:test or another selected runner and CI. 2. Turn the review reproductions into assertions with isolated fixtures and mocked external services. 3. Separate fast deterministic tests from explicitly requested real-agent integration jobs.

Done when. npm test executes assertions and fails on each seeded defect. CI runs on a clean checkout without model credentials. Integration jobs record model versions, timeouts, and costs.

QB-27 The benchmark lets QB define its own scoring target**Critical** | Artifact / code | Phase 4 | Effort LSource: [bench/run.js : 117](#); [bench/run.js : 203](#); [bench/results/T-002_1790654092867.json : 34](#)

Problem. QB generates a contract, exposes it to its own agent, and grades both arms against it. The baseline sees only the original prompt. Wrong or extra generated ACs can reward QB and penalize a correct baseline.

Implement. 1. Freeze human-reviewed task specifications and external check suites before running either arm. 2. Keep grading independent of QB-generated ACs. 3. Use blinded adjudication for ambiguous outcomes and preserve all disagreements.

Done when. Both arms are evaluated against the same original semantic requirement. Correct 30,000 ms -> 30s behavior cannot fail because QB invented 5m. Holdout labels are not revised after viewing arm results.

08 / IMPLEMENTATION TICKETS

Findings 28-30

QB-28 Baseline resources and verification paths are unequal**High** | Code-confirmed | Phase 4 | Effort MSource: [agent/runner.js : 57](#); [bench/baseline.js : 20](#); [bench/run.js : 111](#); [bench/run.js : 206](#)

Problem. QB can receive three ten-minute invocations; baseline gets one eight-minute invocation. Baseline verification receives null context and skips wrapper test execution. The benchmark never calls L5 despite describing a five-layer evaluation.

Implement. 1. Match or explicitly budget resources and give baseline native planning/testing. 2. Use a shared external evaluator for both arms. 3. Run production orchestration through one API and add an explicit memory-off/on experimental condition.

Done when. Run manifests show equal declared budgets and identical external grading. A test invocation spy confirms both arms are graded. The memory experiment proves actual recall calls and records its starting store.

QB-29 Latest-per-task aggregation mixes experiments**High** | Artifact / code | Phase 4 | Effort MSource: [bench/report.js : 32](#); [bench/run.js : 156](#)

Problem. The report replaces older rows by task ID and combines results with different contracts or code versions. Patches, judge evidence, exact model identifiers, and full test results are absent. The latest T-006 artifact has no baseline result.

Implement. 1. Assign immutable experiment/run/arm IDs and pin every relevant version. 2. Store complete evidence bundles and do not overwrite earlier trials. 3. Report missing runs explicitly and compare only matched experimental groups.

Done when. A report rejects mixed versions by default and reproduces exactly from a manifest. Missing baseline rows stay missing rather than becoming losses or successes. Each score links to the evaluated patch and checks.

QB-30 Six tuned utility tasks do not establish effectiveness**High** | Design / artifact | Phase 4 | Effort LSource: [bench/tasks.json : 1](#); [docs/test-runs.md : 134](#)

Problem. The suite is narrow, repeatedly used during development, and contains mostly single-file additions. There is no held-out evaluation, causal ablation, independent verifier calibration, or measurement of human review effort.

Implement. 1. Freeze a diverse development set and untouched holdout across repositories. 2. Run repeated, paired trials and ablations. 3. Report uncertainty, all failures, cost/time, human interventions, and independent correctness rather than internal PASS alone.

Done when. The evaluation protocol is reviewed before execution. Results include repository/task strata and run failures. Improvement claims survive matched baselines and are bounded to the tested task distribution.

08 / IMPLEMENTATION TICKETS

Findings 31-33

QB-31 Supported adapters and onboarding are not productized**Medium** | Code-confirmed | Phase 5 | Effort M

Source: agent/schema.js : 9; qb.js : 28; README.md : 239

Problem. Only Claude Code, manual, and dry-run are implemented. Documentation examples can default to dry-run, while product framing names other agents. There is no explicit runtime compatibility contract or dependable preflight workflow.

Implement. 1. Publish a narrow support matrix and validate all CLI options. 2. Add a doctor command for runtime, agent auth, model availability, Git, sandbox, and check adapters. 3. Version the agent adapter interface and document release/license/support decisions.

Done when. A fresh supported machine completes install, doctor, demo, failure recovery, and artifact inspection from the docs. Unsupported agents/runners and invalid retry counts fail before model execution.

QB-32 Telemetry and privacy descriptions overstate anonymity**High** | Code-confirmed | Phase 5 | Effort M

Source: qb.js : 235; qb.js : 278; landing_page/privacy.html : 135

Problem. Opt-in telemetry contains the registration email and a deterministic 32-bit task hash; it is not anonymous. The privacy page claims source never leaves the machine despite the cloud coding-agent path. Local artifacts may also contain source and task text.

Implement. 1. Write a factual data-flow inventory and correct UI/docs/email wording. 2. Minimize telemetry and use explicit consent, retention, deletion, and redaction. 3. Prefer random run IDs; use keyed identifiers only when justified and keep model-provider flows explicit.

Done when. Consent previews the actual payload. Telemetry is off by default, contains no accidental source/secret data, and supports deletion. Privacy text matches observed agent, model, storage, and API flows.

QB-33 Metrics can be forged with a known registration email**High** | Reproduced locally | Phase 5 | Effort M

Source: landing_page/_worker.js : 88; landing_page/functions/api/metrics.js : 1

Problem. A registered email is the only authorization check. A mocked request without credentials inserted metrics; the string false became passed=1 and negative attempts/duration were accepted. Such data cannot substantiate product effectiveness.

Implement. 1. Issue and validate revocable scoped credentials after registration verification. 2. Require strict request schemas, bounds, run IDs, replay protection, and rate limits. 3. Label client-reported outcomes separately from independently verified benchmark results.

Done when. A known email alone receives 401/403. String booleans, negative values, duplicate run IDs, and oversized payloads are rejected. Valid authenticated records preserve their exact declared types.

08 / IMPLEMENTATION TICKETS

Findings 34-36

QB-34 Admin exports expose avoidable credential and CSV risks**High** | Code / risk | Phase 5 | Effort MSource: [landing_page/_worker.js : 125](#); [landing_page/_worker.js : 160](#); [landing_page/_worker.js : 276](#)

Problem. Admin credentials are accepted in URL query strings, which may enter history and logs. CSV cells are quoted but spreadsheet formula prefixes are not neutralized. Read endpoints return personal signup data without explicit no-store handling.

Implement. 1. Use protected admin authentication and authorization headers, not query secrets. 2. Add no-store and safe export handling. 3. Treat user-controlled CSV cells as text and paginate access; redact credentials from logs.

Done when. Query-string secrets are rejected. Formula-like signup fields open as inert text in supported spreadsheet clients. Admin responses are not cached and unauthorized requests never return signup data.

QB-35 Signup input, delivery, and abuse handling are fragile**High** | Code / local probe | Phase 5 | Effort MSource: [landing_page/_worker.js : 45](#); [landing_page/_worker.js : 78](#); [landing_page/_worker.js : 243](#)

Problem. Null or wrongly typed JSON can throw before validation. No application-level payload bounds or rate limits are present. Select-then-insert deduplication can race. Email failures are settled or returned without durable retry, yet the API reports success.

Implement. 1. Validate types, lengths, and allowed fields before use; add abuse controls. 2. Use atomic insert-on-conflict behavior. 3. Track registration separately from delivery and queue idempotent email retries with observable failures.

Done when. Null, numeric emails, oversized text, duplicate concurrent submissions, and a mocked email-service failure have defined responses. Failed delivery can recover without duplicate signups or repeated welcome messages.

QB-36 Legacy proxy permits unauthenticated paid upstream calls**Critical if deployed** | Reproduced locally | Phase 1 | Effort SSource: [netlify/functions/compile.js : 18](#); [netlify.toml : 6](#)

Problem. The proxy checks authentication only when QB_PROXY_SECRET is set. With a configured upstream key and missing proxy secret, a mocked unauthenticated request reached the upstream service. Live deployment of this route was not inspected.

Implement. 1. Remove the unused proxy or fail closed when required auth configuration is absent. 2. If retained, validate model, payload, token limits, credentials, and quotas. 3. Add deployment checks that reject missing required secrets without printing them.

Done when. Missing or incorrect credentials never call upstream. A local test with a dummy upstream key and no proxy secret returns an explicit configuration/auth error. Confirm actual deployment before assigning exposure status.

08 / IMPLEMENTATION TICKETS

Findings 37-38

QB-37 Deployment and public claims have drifted**High** | Code-confirmed | Phase 5 | Effort M

Source: wrangler.toml : 1; netlify.toml : 1; landing_page/src/north-star-metric.html : 46; README.md : 73

Problem. Workers, Pages functions, and Netlify routes coexist. Netlify API routing points to functions that do not include the signup backend. README and landing-page result claims differ, and the page combines 6/6 with +34pp from earlier experiments.

Implement. 1. Choose one canonical deployment path and remove or clearly archive alternatives. 2. Add deployment smoke checks for static assets and every public API. 3. Generate evidence-linked benchmark claims from approved experiment manifests; keep projections labeled.

Done when. A clean staging deploy passes signup/report/metrics routing tests with mocked mail. Every published metric resolves to a consistent experiment, and supported-agent, cost, and local-processing claims match implementation.

QB-38 Runs lack a durable, versioned evidence record**High** | Code-confirmed | Phase 0 | Effort M

Source: qb.js : 99; qb.js : 156; qb.js : 209; bench/run.js : 175

Problem. Artifacts are optional and spread across layer directories. There is no authoritative run manifest connecting the base snapshot, accepted requirements, attempts, check execution, model settings, and final decision. This prevents diagnosis, replay, and audit.

Implement. 1. Define versioned run, requirement, check, attempt, and evidence schemas. 2. Persist a manifest and append-only events by default in a controlled run directory. 3. Add checksums, redaction, explicit retention, and crash-safe writes; keep migrations documented.

Done when. One run ID retrieves the complete decision trail without relying on console output. A crashed run resumes or terminates explicitly. Replaying verdict aggregation on the stored evidence reproduces the recorded outcome.

09 / DELIVERY ROADMAP

Phases 0-1: establish a safe foundation

The sequence is gate-based. Size S means a contained change, M means several coordinated changes, and L means an architectural or evaluation workstream. These are relative planning sizes, not delivery promises. A junior developer implements; a senior reviewer approves security and decision-policy changes.

Phase 0 - Freeze evidence and establish regression tests

Tickets: QB-26, QB-38. **Dependency:** none. **Owner:** junior developer; technical lead approves schemas.

- Pin the reviewed revision and preserve original benchmark artifacts unchanged. Create a new implementation branch and record the issue register in the team tracker.
- Add real deterministic tests, temporary repository fixtures, mocked model/agent/API adapters, and CI. First encode the false-PASS and safety reproductions so later fixes have observable targets.
- Introduce versioned run/evidence schemas and one run directory. Preserve compatibility intentionally; add migration tests when schema fields change.

Phase 0 exit gate

A clean install runs meaningful tests without cloud credentials. Seeded defects fail assertions. One run manifest connects input, patch, checks, and outcome, including a deliberately interrupted run.

Phase 1 - Contain execution and preserve the full result

Tickets: QB-01 to QB-05, QB-22, QB-36. **Dependency:** Phase 0. **Owner:** junior developer with senior security review.

- Remove shell interpolation and untrusted metadata writes first. Disable or close the optional legacy proxy before connecting credentials to it.
- Build the disposable checkout/sandbox boundary, minimal environment policy, full snapshot capture, and explicit agent result types. Treat benchmark reset as fixture creation.
- Verify preservation of the source working tree, all change types, process cleanup, and policy-denied actions. Keep human merge review in place.

Phase 1 exit gate

Host-write and forbidden-egress fixtures are denied. Original user files survive every success/failure path. New, staged, and committed changes are inspectable. Execution failures are never reported as dry-runs.

10 / CONTRACT AND VERIFIER CORRECTNESS

Phase 2: make acceptance mean something

Tickets: QB-06 to QB-17. **Dependency:** safe execution, real regression tests, and agreed evidence schemas. This is the core product work; it merits the largest engineering allocation.

Order	Work package	Review requirement
2A	Strict contract union: finalized vs needs_clarification. Preserve source clauses; prohibit metadata overrides and silent defaulting of incomplete semantics.	Lead reviews requirement traceability and blocking behavior.
2B	Ground intent in a lightweight repo survey. Validate generated examples and material assumptions. Add missing/extra requirement checks.	Founder or task owner approves changed meaning.
2C	Implement the first protected check adapter with exit codes, machine reports, baseline/candidate comparisons, and explicit error states.	Senior reviews test ownership and sandbox boundary.
2D	Aggregate outcomes with hard policy/check gates. Give failures, errors, and unknowns distinct recovery paths.	Table-driven tests cover all state combinations.
2E	Use strict judge output, criterion-specific evidence, and syntax-aware structural checks. Keep LLM opinions subordinate to evidence.	Review attack fixtures and evidence references.
2F	Calibrate preservation handling and vote count. Detect repeated unchanged patches; cap retries and evidence collection.	Evaluate against independent patch labels.

Required demonstrations

- T-002: a correct duration converter passes the independent checks, and each incorrect numerical example is rejected before execution.
- T-005: a constant-zero stub fails; counters respond correctly to controlled events and separate instances.
- A crashed runner, malformed judge output, empty contract, forbidden edit, and incomplete evidence each produce the correct non-PASS state.
- A test failure is repaired from its actual failure evidence. An unknown preservation claim triggers a relevant check or remains unresolved.

Phase 2 exit gate

All mandatory adversarial regression fixtures pass, every required clause maps to checks, and no required unknown/error receives VERIFIED. A senior reviewer can trace every approval from original request to executed evidence.

11 / RETRIEVAL, RECOVERY, AND PERFORMANCE

Phase 3: improve context and memory reliably

Tickets: QB-18 to QB-21, QB-23 to QB-25. **Dependency:** stable contracts and decision semantics. Optimize measured bottlenecks after acceptance is trustworthy.

Implementation order

- **3A - Ground the index:** parse supported syntax, qualify symbols, record source hashes, build bounded import/caller edges, and distinguish test associations from measured coverage.
- **3B - Retrieve with a budget:** combine explicit paths, symbol/content matches, dependencies, and test relationships. Expose low-confidence retrieval and refresh modified files on retries.
- **3C - Fix repair learning:** persist attempt transitions, then associate a repair with its changed patch and reevaluated evidence. Keep hints labeled by outcome quality.
- **3D - Harden the store:** inject configuration before construction, prevent namespace collisions, retain negation, validate paths/revisions, lock writes, and support retention/compaction.
- **3E - Bound the runtime:** asynchronous execution, per-stage deadlines, whole-run budgets, cancellation, and bounded local-model concurrency. Measure optional vote count before spending more latency on it.

Suggested benchmark fixtures for this phase

Fixture	Required observation
200+ source files; target symbol in a generic filename	Relevant implementation, callers, and checks are retrieved without depending on filename overlap alone.
Two same-named exports; trailing CJS export; late-file export	Correct qualified symbols and source spans survive indexing.
Opposite instructions; moved file; changed repo revision	No automatic reuse of stale or contradictory actionable repair advice.
Concurrent writes; truncated JSONL record; large history	Counts are preserved, corruption is reported, and recall latency stays within the declared support target.
Stalled model and cancelled agent with children	A bounded terminal state, saved evidence, and no surviving controlled child processes.

Phase 3 exit gate

The supported-size fixture set passes, recall cannot confer authority, and p50/p95 latency and resource use are recorded. Choose numeric service targets from pilot hardware measurements; do not invent performance guarantees.

12 / INDEPENDENT EVALUATION

Phase 4: run an experiment you can defend

Tickets: QB-27 to QB-30. **Dependency:** frozen Phase 2 semantics and Phase 3 behavior. A professor or other independent reviewer should approve the protocol before holdout execution.

Build the evaluation in this order

- Select roughly 30-50 development tasks from several repositories to debug the harness. Freeze a separate untouched holdout. Cover additions, bug fixes, integrations, refactors, state, errors, and already-satisfied requests.
- Before either arm runs, establish accepted task meaning, external checks, base snapshots, and adjudication rules. Restrict hidden evaluation artifacts from the coding agent.
- Pin all versions and configurations. Start each arm in an equivalent disposable environment. Record model warm/cold conditions, resource budgets, and every failed or missing run.
- Use paired runs with repeated trials where feasible. Randomize execution order. Three trials can be an initial protocol choice; select repetitions and sample size based on variance and the effect you want to detect.

Arm	What it isolates
A: competent native agent	Normal instructions, native planning and testing; declared resource budget.
B: A + simple retry loop	Whether deterministic test feedback and equal retry budget explain the improvement.
C: B + accepted QB contract	Incremental value from intent specification.
D: C + QB context	Incremental value from context selection.
E: D + QB verifier feedback	Incremental value from the custom verification and repair policy.
F: E + memory	Value on later unseen tasks from a controlled prior-task sequence.

All arms use the same external final grader. The generated contract never becomes the hidden scoring authority. For memory, separate training/experience from evaluation and retain a clean memory-off control.

Phase 4 exit gate

A third party can reproduce each result from a manifest and patch. Report cost and correctness together, confidence intervals, exclusions, and ablations. If gains vanish, narrow or revise the product hypothesis before broad release.

13 / DELIVERY AND OPERATIONS

Phase 5: qualify a supervised beta

Tickets: QB-31 to QB-35, QB-37. **Dependency:** earlier gates and an independently reviewed evaluation. Website safety fixes may proceed earlier; beta readiness is still gated by the core verifier.

Workstream	Required deliverable
Install and support	Versioned release, supported runtime/agent/check matrix, doctor command, clear examples, recovery guide, and a documented license/support decision.
Website/API	One canonical deployment, schema validation, scoped authentication, rate limits, bounded requests, safe admin exports, idempotent signup, and observable mail delivery.
Data handling	Accurate privacy/data-flow explanation, explicit opt-in telemetry, minimal identifiers, retention/deletion process, redaction, and no credentials in query strings.
Operations	Structured logs and run IDs, error categories, timeout/cancel behavior, health checks, migration/rollback procedure, and a named incident owner.
User experience	Inspect accepted assumptions before execution, understand progress, inspect the complete patch/evidence, see unresolved items, and stop or resume safely.
Pilot measurement	Recruit 3-5 consenting design partners with supported repositories. Record review time, corrections, independent correctness, cost, and user-reported friction.

Rules for the first pilot

- Every merge remains human-reviewed. Limit task scope and isolate execution. Do not grant production deployment authority as part of this pilot.
- Record every intervention, including clarification, manual edits, reruns, and environment repair. Obtain consent before retaining task content or source.
- Triage false approvals immediately. Preserve evidence, suspend affected automatic approvals, fix the mechanism, and rerun the relevant regression and evaluation suites.

Phase 5 exit gate

Fresh users complete onboarding and recovery without the founder operating their machine. No open critical safety/false-approval findings remain in supported scope. Pilot evidence demonstrates useful review-effort reduction at the agreed quality and cost thresholds.

14 / TESTS THAT MUST CATCH THE OLD FAILURES

Minimum acceptance-test matrix

Add these to CI with controlled fixtures. They are minimum regression categories, not a substitute for the done-when conditions in all 38 tickets. Real-agent tests should be separate, budgeted jobs.

Case	Expected result	Tickets
Filename with shell substitution	Literal path; no side effect	01
Model-supplied traversal ID	Trusted ID retained; safe save path	04
Empty/clarification-only contract	BLOCKED; agent never called	08, 17
Incorrect conversion examples	Rejected or explicitly unresolved	13
Constant-zero VAD implementation	Behavioral check FAIL	14, 16
Runner exits 2 / times out / collects none	ERROR or explicit blocked policy; never PASS	06
Non-JSON negative judge response	Invalid judgment; never guessed true	07
Failing test plus unknown AC	FAILED with both pieces of evidence	10
Forbidden path plus all-positive judge	FAILED policy check	09
New / staged / agent-committed files	Complete captured changes	03
Missing later hunk or unchanged helper	Retrieve evidence or UNRESOLVED	11
Comment-only target and unrelated return	No structural/behavioral proof	12
Two-attempt successful repair	Linked history and repair record	23
Repo-key collision and opposite goal	Separate stores; no blind repair reuse	24
Known email, no telemetry credential	401/403; no insert	33
String false and negative counts	Schema rejection	33, 35
Missing proxy secret with dummy key	No upstream call	36
Dirty user checkout before benchmark	Preserved byte-for-byte	05
Cancelled execution with partial changes	Saved evidence; no uncontrolled children	21, 22

Test ownership: the junior developer writes a failing regression before each fix. A reviewer confirms it exercises the failure mechanism, not a restatement of implementation. Close the ticket only with the test, evidence, and documentation changes linked.

15 / PRODUCT QUALIFICATION METRICS

Measure correctness and user effort separately

Metric	Definition and interpretation
Independent task success	Tasks satisfying the frozen external requirements / all assigned tasks. Report infrastructure failures separately, without silently removing them.
False acceptance	Incorrect candidates labeled VERIFIED / all independently labeled incorrect candidates. This tests verifier safety, not agent success.
Approval precision	Correct candidates among those labeled VERIFIED. Report the dataset class balance; precision changes with the incoming defect rate.
Abstention / resolution coverage	Fraction unresolved and fraction receiving a supported decision. A verifier that never approves needs a different assessment from one that approves everything.
Human interventions per completed task	Clarifications, manual corrections, reruns, and environment repairs required per independently completed task. Retain incomplete tasks and report their effort too.
Review time and total cost	Human review/correction minutes, wall time p50/p95, coding-agent usage, local compute, and verification overhead. Local inference does not imply zero total cost.
Repair effectiveness	Externally incorrect initial candidates corrected by the repair / incorrect initial candidates. Verify the patch changed meaningfully and track regressions introduced by repair.

Proposed qualification thresholds

- **Engineering gate:** all mandatory safety/state tests pass; no open critical issue in supported scope; complete evidence for every approval; clean install and recovery demonstrations.
- **Verifier gate:** publish results on a reviewed incorrect-patch set, including omitted requirements, tampered tests, prompt injection, and integration defects. Set the acceptable false-acceptance target with the professor and pilot users before measuring it.
- **Value gate:** independently measured correctness is acceptable and review/correction effort improves against the competent baseline within the agreed resource budget.

Zero observed errors is not proof of zero risk

For an illustrative fixed set of 100 independent representative incorrect patches, zero false approvals gives a one-sided 95% binomial upper bound near 3%. Correlated or curated cases weaken that interpretation. Report task/repository clustering and uncertainty; keep human merge review.

16 / PROFESSOR REVIEW PREPARATION

Questions you should expect to be asked

Question	Evidence-based answer and next step
What exactly does verified mean?	Within a declared support scope, every required check and policy passed with linked evidence. It does not prove every possible behavior. Show the outcome definitions and unresolved path.
Who verifies the specification?	The current compiler can invent or omit requirements. Show T-002 and T-005 openly, then show provenance, computable-example checks, and the accepted-contract gate.
Why should three 7B votes be trusted?	They should not be treated as independent proof. Propose an equal-cost calibration of one versus three calls, with external labels and abstention metrics.
Is your baseline deliberately weak?	The old harness confounded instructions, retry budgets, and grading. Present the matched baseline and ablation protocol; do not defend the old headline as established effectiveness.
Can the agent game the tests?	Currently the writable workspace permits it. Demonstrate the protected test/policy boundary and a deliberately tampered test that cannot authorize acceptance.
What happens when nothing can be verified?	Return UNRESOLVED with the missing evidence and the next check or human decision. Do not convert uncertainty into success.
What is the research contribution?	A hypothesis about reducing human interventions through explicit requirements and independent evidence across agents. The contribution must be established by controlled experiments, not layer count.
Why not just prompt Claude better?	That is a required baseline. If an equal-budget native agent or simple test-retry wrapper matches QB, narrow the product to the incremental capability that survives the comparison.

Specific feedback to request

- Is the independent oracle strong enough, and which task classes should form the holdout?
- Which ablations isolate the causal claim most economically? How should repeated tasks and repository clustering be handled?
- What false-acceptance and review-effort evidence would justify a supervised beta in the chosen scope?

Bring the failure evidence and the proposed protocol. A candid account of what is broken, paired with reproducible fixes, is more credible than a higher percentage from the same six tasks.

17 / REVIEW PACKET AND DIFFERENTIATION

Take a controlled demonstration to the meeting

Demonstrate this sequence after the relevant phase gates pass. Use a disposable fixture repository and a pre-recorded fallback if local model startup is unreliable.

Step	What to show
1. Request fidelity	A natural-language task, preserved source clauses, the accepted contract, and one proposed assumption requiring an explicit decision.
2. Independent failure	A plausible incomplete implementation that existing tests miss. The task-specific protected check rejects it with exact evidence.
3. Repair	A bounded repair prompted by that concrete failure. Compare candidate hashes and show the check changing from fail to pass.
4. Honest uncertainty	A runtime property with no available test adapter. QB returns UNRESOLVED and names the missing evidence.
5. Reproducibility	One run ID opens the full patch, versions, executed commands, results, votes, costs, and final decision; a replay reproduces aggregation.

Competitive overlap to acknowledge

Official documentation already describes repository search, rules, command execution, browser testing, and checkpoints in Cursor; structured application testing and recordings in Devin; and configurable execution environments and history processing in SWE-agent. Those broad capabilities are not a defensible moat by themselves. [E1-E3]

A potential differentiator is a portable acceptance layer with reliable requirement traceability, protected checks, calibrated unknowns, and low review effort across tools. That remains a hypothesis until external trials establish it. A consented dataset of requirements, failure mechanisms, repairs, and independently confirmed outcomes could become a durable asset.

Bring this review packet

- Pinned source revision, current issue status, completed gate evidence, architecture/threat model, and one complete successful plus one unresolved run.
- Frozen evaluation protocol, task selection rationale, competent baseline, ablations, and a table of all runs with uncertainty and cost.
- Decisions you need from the professor: experimental design, oracle independence, scope boundaries, and acceptable residual uncertainty.

Claim you can make now

Quarterback is an experimental control loop. The review found concrete correctness and safety failures. The team has a staged plan to make acceptance evidence-based and to test whether that reduces human review effort. Do not describe planned gates as completed.

18 / LOCAL REVIEW RECIPES

Reproduce the important failures safely

Use only temporary fixture repositories and mocked services. Do not run destructive benchmark reset logic against a working project. These recipes describe the reviewed behavior; the expected post-fix behavior is in each ticket.

Recipe A - Empty contract reaches PASS (QB-08)

```
const { verify } = require('./verify/verifier');
verify(
  { id: 'probe', clarifying_question: 'Which behavior?' },
  null,
  { id: 'probe-run', diff: 'diff --git a/x b/x\n+x' },
  { repoPath: null }
).then(r => console.log(r.verdict));
// Reviewed result: pass. No model call is needed.
```

Recipe B - Negative prose becomes three positive votes (QB-07)

```
global.fetch = async () => ({
  ok: true,
  json: async () => ({ message: {
    content: 'The criterion is not implemented.'
  } })
});
const { judgeAll } = require('./verify/judge');
judgeAll(
  [{ id: 'AC-1', criterion: 'targetFunction exists' }],
  'diff --git a/x b/x\n+// fixture', {}
).then(console.log);
// Reviewed result: met=true; votes=[true,true,true].
```

Recipe C - Test exit code is lost (QB-06)

Create a temporary package.json whose test command is `node -e "process.exit(2)"`. Call `runChecks` with `test_runner=node-test`. The reviewed result is zero passed and zero failed tests. Feed controlled affirmative judge responses into `verify()`: it can return PASS. No real model is involved.

Recipe D - API validation and authentication (QB-33, QB-36)

Import the Worker locally and supply a fake D1 object whose registration lookup returns a test user. Send a synthetic metrics Request without credentials using `passed="false"`, `attempts=-7`, and `duration_ms=-100`. The reviewed code inserts `passed=1` and the negative counts. For the proxy, use a dummy upstream key, omit the proxy secret, and stub fetch; an unauthenticated request reaches the stub.

The command-execution reproduction used a harmless marker in an isolated temporary repository. A security regression should use the same containment and assert that no marker is created. Do not test these conditions against the live signup, mail, or metrics endpoints.

19 / REFERENCES AND HANDOFF

Source map and completion record

Authoritative source snapshot: [arshad8049/quaterback @ 4ecc6a7](#)

Full revision: 4ecc6a7993d41c12f92b227eb7cb9af7f8efa60f

Review date: 1 October 2026. All ticket source links point to this immutable revision.

Source group	Where to inspect
Runtime / evidence	qb.js; agent/runner.js; agent/schema.js; agent/briefing.js. Run-state flow, agent invocation, change capture, and artifacts.
Intent / context	intent/compiler.js; intent/schema.js; context/builder.js; context/extractor.js; context/git.js; context/detector.js.
Verification / memory	verify/checker.js; verify/judge.js; verify/verifier.js; verify/schema.js; memory/index.js; memory/store.js; memory/scorer.js.
Benchmark	bench/run.js; bench/baseline.js; bench/report.js; bench/tasks.json; all 15 bench/results/*.json files; docs/test-runs.md.
Delivery / API	landing_page/_worker.js; landing_page/functions/api/; landing_page/privacy.html; landing_page/schema.sql; netlify/functions/compile.js; deployment configs.

External primary sources

[E1] [Cursor: Agent overview](#) - agent tools, browser interaction, and checkpoints. Accessed 1 October 2026.

[E2] [Devin: Testing and Video Recordings](#) - application test workflow and recordings. Accessed 1 October 2026.

[E3] [SWE-agent: Architecture](#) - environment, tools, history processing; documentation directs users toward mini-swe-agent. Accessed 1 October 2026.

Ticket closure template

Field	Required entry
Implementation	Issue ID, owner, PR/commit, affected files, migration and compatibility notes.
Evidence	Failing pre-fix reproduction, passing regression, CI run, relevant real integration evidence, and updated documentation.
Review	Senior signoff for safety/acceptance policy, independent reviewer for evaluation changes, remaining limitations, and closure date.

Use this as a versioned work plan

As fixes land, update issue status and link evidence in the team tracker. Preserve this review as the baseline. Reassess readiness against phase gates and supported scope rather than counting closed tickets alone.